# Casamento de Cadeias em Sequências Genômicas: Fundamentos Matemáticos, Engenharia e Análise Assintótica dos Algoritmos de Rabin-Karp e Knuth-Morris-Pratt

Arthur de Oliveira Mendonça
Italo Xavier Fonseca
Thiago Augusto Silva Martins
Thales Guerreiro
Matheus Tavares

*Departamento de Computação (DECOM-DV) – CEFET-MG Divinópolis*

Disciplina: Algoritmos e Estruturas de Dados II / Projeto e Análise de Algoritmos
Professor: Prof. Dr. Michel Pires da Silva

Novembro de 2026

**Resumo**

A localização exata de subcadeias no interior de grandes volumes textuais constitui um problema clássico da ciência da computação, cuja relevância prática se intensificou com o sequenciamento genômico em larga escala. Neste relatório técnico, investigamos o paradigma de casamento de cadeias (*string matching*) sob a perspectiva teórica do Capítulo 32 de Cormen et al. (CLRS) e sob o ponto de vista empírico da bioinformática. Formalizamos a aritmética modular de polinômios via Regra de Horner no algoritmo de Rabin-Karp e a teoria de autômatos implícitos baseada na função de prefixo $\pi$ do algoritmo de Knuth-Morris-Pratt (KMP). Demonstramos analiticamente as cotas assintóticas $O$, $\Omega$ e $\Theta$ de tempo e memória, incluindo a prova amortizada de linearidade $\Theta(n + m)$. Por fim, avaliamos uma implementação de referência em C++17 aplicada ao genoma completo do vírus SARS-CoV-2 (`NC_045512.2`) e ao

transcrito humano *TP53* (`NM_000546.6`), discutindo o papel do casamento exato na etapa de semeação (*seeding*) do alinhamento genético e os gargalos de arquitetura de processadores modernos.

# Índice


**1 Introdução** **3**

**2 Fundamentação Teórica** **4**

2.1 Formalização do Problema e Notações de Cadeias . . . . . 4

2.2 Aritmética Modular e *Rolling Hash* no Algoritmo de Rabin-Karp . . . . . 5

2.3 A Função de Prefixo $\pi$ e o Teorema Central do KMP . . . . . 6

2.4 Implicações em Bioinformática: Do Casamento Exato ao Alinhamento Genético . . . . . 7

**3 O Paradigma Algorítmico (Engenharia da Solução)** **8**

3.1 Pseudocódigos Formais (`algorithm2e`) . . . . . 8

3.2 Diagramas Visuais Passo a Passo . . . . . 11

**4 Análise Assintótica Rigorosa (Custos)** **12**

4.1 Complexidade da Busca Ingênua . . . . . 12

4.2 Complexidade do Algoritmo de Rabin-Karp . . . . . 13

4.3 Complexidade Amortizada Rigorosa do Algoritmo KMP . . . . . 14

**5 Estudo de Caso e Implementação Prática** **15**

5.1 Arquitetura do Repositório e Ambiente de Testes . . . . . 15

5.2 Experimento 1: Busca de Padrões Biológicos no Genoma do SARS-CoV-2 . 16

5.3 Experimento 2: Escalabilidade Linear com Entradas Progressivas ($N$) . . . 17

5.4 Experimento 3: Pior Caso Patológico em DNA Repetitivo e Impacto do Primo $q$ . . . . . 18

5.5 Discussão Crítica: Gargalos de Hardware e de Linguagem em C++17 . . . 19

**6 Conclusões** **20**

**Apêndice: Cheat Sheet (Guia de Consulta Rápida)** **23**

# 1 Introdução

A manipulação eficiente de cadeias de caracteres sustenta desde ferramentas de busca em editores de código e sistemas de detecção de intrusão em redes até a montagem e anotação de genomas inteiros. Em termos computacionais, o problema consiste em identificar todas as posições onde um padrão $P$ de comprimento $m$ ocorre de maneira contígua dentro de um texto $T$ de comprimento $n$, com $n \geq m$. Quando $n$ se restringe a poucos milhares de caracteres, mesmo uma verificação exaustiva posicional responde em frações de milissegundo. O cenário muda radicalmente na bioinformática moderna. Um único cromossomo humano contém dezenas de milhões de nucleotídeos, e um banco de referência genômica acumula bilhões de bases nitrogenadas representadas sobre o alfabeto quaternário $\Sigma = \{\mathrm{A}, \mathrm{C}, \mathrm{G}, \mathrm{T}\}$.

Historicamente, a abordagem direta para localizar um padrão baseia-se no algoritmo ingênuo (ou de força bruta), que desliza o padrão uma posição por vez sobre o texto e compara os caracteres da esquerda para a direita. Essa estratégia padece de uma falha estrutural severa: ela ignora toda a informação obtida nas comparações parciais anteriores. Sempre que ocorre uma divergência no índice $j$ do padrão, o ponteiro do texto retrocede para $s+1$ e o processo recomeça do zero, levando ao custo de pior caso $\Theta((n-m+1)m)$. Em textos de linguagem natural, cujo alfabeto possui dezenas de símbolos, uma falha costuma ocorrer logo na primeira ou segunda letra. No DNA, entretanto, o alfabeto reduzido de apenas quatro letras eleva a probabilidade de coincidência acidental entre duas bases para 25%, além de genomas reais conterem extensas regiões repetitivas, como caudas poli-A e microssatélites, que forçam a degradação quadrática da busca ingênua.

Paralelamente, na biologia computacional, o alinhamento global ou local de sequências com mutações e lacunas (*gaps*), resolvido pelos algoritmos de programação dinâmica de Needleman-Wunsch e Smith-Waterman, exige tempo $\Theta(nm)$. Executar uma matriz de programação dinâmica contra os $3,2 \times 10^9$ pares de base do genoma humano para cada leitura de sequenciamento é computacionalmente proibitivo. Para contornar essa barreira, os alinhadores genéticos reais recorrem aos algoritmos lineares de casamento exato estudados neste trabalho: utiliza-se o *rolling hash* de Rabin-Karp ou autômatos derivados do KMP para localizar rapidamente pequenas sementes exatas (*seeds* ou $k$-mers) em tempo linear, restringindo o alinhamento caro por programação dinâmica apenas às regiões candidatas.

O objetivo deste trabalho é investigar, formalizar matematicamente e validar experimentalmente os algoritmos de Rabin-Karp e Knuth-Morris-Pratt (KMP), contrastando seu desempenho teórico e prático frente à busca ingênua em sequências genômicas reais e sintéticas. Buscamos demonstrar não apenas as provas formais de complexidade assintótica do Capítulo 32 de Cormen et al. [1], mas também expor como fatores de baixo nível de

hardware (como o custo em ciclos de clock da divisão inteira na ALU e a localidade espacial na memória cache L1) impactam o tempo real de execução em C++17.

O restante deste documento está organizado em cinco capítulos adicionais e um apêndice. A Seção 2 desenvolve a fundamentação matemática, cobrindo a aritmética modular do Rabin-Karp, o Lema da Superposição de Sufixos e a estrutura da função de prefixo do KMP. A Seção 3 apresenta a engenharia dos algoritmos por meio de pseudocódigos no padrão `algorithm2e` e diagramas visuais passo a passo. A Seção 4 conduz a análise assintótica rigorosa ($O$, $\Omega$ e $\Theta$) de tempo e memória. A Seção 5 detalha o estudo de caso em bioinformática com sequências do NCBI (`NC_045512.2` e `NM_000546.6`), os experimentos de estresse e a análise de gargalos de hardware. A Seção 6 sintetiza as conclusões sobre onde o paradigma é aplicável e onde ele falha, seguida pelo Apêndice contendo o *Cheat Sheet* de consulta rápida.

# 2 Fundamentação Teórica

## 2.1 Formalização do Problema e Notações de Cadeias

Seja $\Sigma$ um alfabeto finito de símbolos. Na análise genômica, adotamos $\Sigma = \{\mathrm{A}, \mathrm{C}, \mathrm{G}, \mathrm{T}\}$, de cardinalidade $d = |\Sigma| = 4$, ou sua representação em bytes ASCII onde $d = 256$. Definimos o texto como um vetor $T[1..n]$ de comprimento $n$ e o padrão buscado como um vetor $P[1..m]$ de comprimento $m$, com $m \leq n$ (adotamos nas demonstrações teóricas a indexação base-1 do livro-texto de Cormen et al. [1], explicitando na Seção 5 a transição para base-0 na linguagem C++).

**Definição 2.1** (Deslocamento Válido e Inválido)**.** *Dizemos que o padrão $P$ ocorre com deslocamento (shift) $s$ no texto $T$, onde $0 \leq s \leq n - m$, se e somente se $T[s+1..s+m] = P[1..m]$, isto é, $T[s+j] = P[j]$ para todo $j \in \{1, 2, \ldots, m\}$. Se essa igualdade se verifica, $s$ é chamado de **deslocamento válido**; caso contrário, $s$ é um **deslocamento inválido**.*

Denotamos por $\Sigma^*$ o conjunto de todas as cadeias finitas formadas por caracteres de $\Sigma$, incluindo a cadeia vazia $\varepsilon$ de comprimento zero. Dados $w, x \in \Sigma^*$, escrevemos $w \sqsubset x$ para indicar que $w$ é um **prefixo** de $x$ (existe $y \in \Sigma^*$ tal que $x = wy$) e escrevemos $w \sqsupset x$ para indicar que $w$ é um **sufixo** de $x$ (existe $y \in \Sigma^*$ tal que $x = yw$). Definimos ainda $P_q = P[1..q]$ como o prefixo de $P$ contendo seus primeiros $q$ caracteres, com $P_0 = \varepsilon$.

Um resultado estrutural indispensável para demonstrar a corretude do algoritmo KMP é o Lema da Superposição de Sufixos:

**Lema 2.1** (Lema 32.1 de Cormen et al. – Superposição de Sufixos)**.** *Sejam $x, y, z \in \Sigma^*$ cadeias tais que $x \sqsupset z$ e $y \sqsupset z$.*

1. *Se* $|x| \leq |y|$, *então* $x \sqsupset y$.

2. *Se* $|x| \geq |y|$, *então* $y \sqsupset x$.

3. *Se* $|x| = |y|$, *então* $x = y$.

*Demonstração.* Se $x \sqsupset z$ e $y \sqsupset z$, então existem cadeias $u, v \in \Sigma^*$ tais que $z = ux = vy$. Suponha $|x| \leq |y|$. Como $x$ e $y$ coincidem exatamente nos últimos $|x|$ caracteres de $z$, podemos decompor $y$ em $y = wx$, onde $w$ compreende os primeiros $|y| - |x|$ caracteres de $y$. Pela definição de sufixo, conclui-se imediatamente que $x \sqsupset y$. Os casos $|x| \geq |y|$ e $|x| = |y|$ seguem por simetria direta. □

## 2.2 Aritmética Modular e *Rolling Hash* no Algoritmo de Rabin-Karp

Proposto por Karp e Rabin [3], o algoritmo de Rabin-Karp substitui a comparação direta de subcadeias pela comparação de assinaturas numéricas inteiras. Assumindo que cada caractere $c \in \Sigma$ seja biunivocamente mapeado para um dígito em $\{0, 1, \ldots, d-1\}$, podemos interpretar qualquer subcadeia de comprimento $m$ como um número inteiro escrito na base posicional $d$.

Dessa forma, o valor numérico $p$ correspondente ao padrão $P[1..m]$ e o valor $t_s$ correspondente à subcadeia $T[s+1..s+m]$ na janela de deslocamento $s$ são definidos pelos polinômios:

$$p = \sum_{i=1}^{m} P[i] \cdot d^{m-i}, \qquad t_s = \sum_{i=1}^{m} T[s+i] \cdot d^{m-i} \tag{1}$$

Pela **Regra de Horner**, ambos $p$ e $t_0$ podem ser avaliados em tempo linear $\Theta(m)$ sem necessidade de exponenciação explícita a cada termo:

$$p = P[m] + d\Big(P[m-1] + d\big(P[m-2] + \cdots + d(P[2] + d \cdot P[1]) \ldots\big)\Big) \tag{2}$$

A eficiência do algoritmo decorre de podermos calcular $t_{s+1}$ a partir de $t_s$ em tempo constante $\Theta(1)$. Ao deslocar a janela de $s$ para $s+1$, removemos o termo de maior ordem $T[s+1] \cdot d^{m-1}$, multiplicamos o resultado pela base $d$ para elevar o grau dos $m-1$ dígitos remanescentes e somamos o novo dígito menos significativo $T[s+m+1]$:

$$t_{s+1} = d\left(t_s - T[s+1] \cdot d^{m-1}\right) + T[s+m+1] \tag{3}$$

Quando $m$ é moderadamente grande, os valores exatos de $p$ e $t_s$ excedem a capacidade dos registradores nativos da CPU (64 bits). Para manter cada operação aritmética em

$\Theta(1)$, todos os cálculos são efetuados no anel de inteiros módulo um número primo $q$ ($\mathbb{Z}_q$), escolhido de modo que $d \cdot q$ caiba confortavelmente em uma palavra de máquina. Definindo a constante pré-computada $h \equiv d^{m-1} \pmod{q}$, a recorrência modular do *rolling hash* assume a forma:

$$t_{s+1} = \Big(d\big(t_s - T[s+1] \cdot h\big) + T[s+m+1]\Big) \bmod q \tag{4}$$

A congruência $t_s \equiv p \pmod{q}$ é condição necessária, mas não suficiente, para que $T[s+1..s+m] = P[1..m]$. Se $t_s \not\equiv p \pmod{q}$, descartamos o deslocamento $s$ imediatamente em $\Theta(1)$. Se $t_s \equiv p \pmod{q}$, pode tratar-se de um casamento verdadeiro ou de um **acerto espúrio** (*spurious hit*, uma colisão de hash modular), exigindo a verificação explícita caractere a caractere.

## 2.3 A Função de Prefixo $\pi$ e o Teorema Central do KMP

O algoritmo desenvolvido por Knuth, Morris e Pratt [2] elimina completamente a possibilidade de acertos espúrios e evita qualquer retrocesso do ponteiro de leitura no texto $T$. Suponha que, no deslocamento atual $s$, os primeiros $q$ caracteres do padrão tenham casado com sucesso, isto é, $P[1..q] = T[s+1..s+q]$, mas ocorra uma divergência no caractere seguinte ($P[q+1] \neq T[s+q+1]$).

Em vez de testar cegamente $s+1$, observamos que os últimos $q$ caracteres lidos do texto são exatamente conhecidos, pois formam o prefixo $P_q$. Se um novo deslocamento $s' = s + (q-k)$ (com $k < q$) for válido, então os primeiros $k$ caracteres do padrão $P_k$ precisam coincidir com os últimos $k$ caracteres de $T[s+1..s+q]$, que são justamente um sufixo de $P_q$. Isso motiva a definição formal da função de prefixo:

**Definição 2.2** (Função de Prefixo $\pi$)**.** *Dado um padrão $P[1..m]$, a função de prefixo para $P$ é a aplicação $\pi : \{1, 2, \ldots, m\} \to \{0, 1, \ldots, m-1\}$ tal que $\pi[q]$ é o comprimento do maior prefixo de $P$ que é um sufixo próprio de $P_q$:*

$$\pi[q] = \max\{k : k < q \ \textit{e}\ P_k \sqsupset P_q\} \tag{5}$$

Para construir a tabela $\pi$ eficientemente usando ela própria, definimos o conjunto de todos os comprimentos de prefixos de $P$ que são sufixos próprios de $P_q$ por meio da aplicação iterada $\pi^{(u)}[q]$ (onde $\pi^{(1)}[q] = \pi[q]$ e $\pi^{(u)}[q] = \pi[\pi^{(u-1)}[q]]$):

$$\Pi^*(q) = \big\{\pi^{(1)}[q], \pi^{(2)}[q], \ldots, \pi^{(t)}[q] = 0\big\} \tag{6}$$

**Teorema 2.1** (Lema da Iteração da Função de Prefixo e Teorema 32.1 de Cormen et al.)**.** *Para todo $q \in \{1, \ldots, m\}$, o conjunto $\Pi^*(q)$ contém exatamente todos os inteiros $k < q$ tais que $P_k \sqsupset P_q$. Ademais, para $q > 1$, se definirmos $E_{q-1} = \{k < q-1 : P_k \sqsupset P_{q-1} \text{ e } P[k+1] = P[q]\}$, então:*

$$\pi[q] = \begin{cases} 0 & \text{se } E_{q-1} = \emptyset, \\ 1 + \max(E_{q-1}) & \text{se } E_{q-1} \neq \emptyset. \end{cases} \tag{7}$$

*Demonstração.* Primeiramente, se $i \in \Pi^*(q)$, então pela transitividade da relação $\sqsupset$ tem-se $P_i \sqsupset P_q$ com $i < q$. Reciprocamente, seja $k < q$ qualquer comprimento tal que $P_k \sqsupset P_q$. Pela definição maximal de $\pi[q]$, sabemos que $k \leq \pi[q]$ e que ambos $P_k \sqsupset P_q$ e $P_{\pi[q]} \sqsupset P_q$. Aplicando o Lema 2.1 (Superposição de Sufixos), segue obrigatoriamente que $P_k \sqsupset P_{\pi[q]}$. Se $k = \pi[q]$, então $k \in \Pi^*(q)$; se $k < \pi[q]$, repetimos o argumento indutivamente sobre $\pi[\pi[q]]$ até alcançar $k$.

Para a segunda parte, note que $P_r \sqsupset P_q$ para $r > 0$ se e somente se $P[r] = P[q]$ e $P_{r-1} \sqsupset P_{q-1}$. Fazendo $k = r - 1$, vemos que $r - 1 \in E_{q-1}$. Portanto, encontrar o maior $r$ que satisfaz $P_r \sqsupset P_q$ equivale a percorrer os candidatos $k \in \Pi^*(q-1)$ em ordem decrescente $(k = \pi[q-1], \pi[\pi[q-1]], \ldots)$ até encontrar o primeiro índice para o qual $P[k+1] = P[q]$, somando 1 ao valor encontrado. □

## 2.4 Implicações em Bioinformática: Do Casamento Exato ao Alinhamento Genético

Na prática bioinformática, duas tarefas distintas costumam ser confundidas: a **busca exata de motivos/iniciadores** e o **alinhamento de sequências homólogas**.

1. **Busca Exata de *Primers*, Sítios de Restrição e *Motifs* Consenso:** No desenho de iniciadores de PCR (*primers* de 18 a 25 pb), na busca de sítios de clivagem enzimática ou na localização de caixas promotoras conservadas (como a TATA-box `TATAAA`), exige-se correspondência exata sobre o cromossomo. O algoritmo KMP resolve essa tarefa em tempo determinístico $\Theta(n + m)$, sendo imune às regiões de baixa complexidade composicional do DNA.

2. **Filtragem e *Seeding* no Alinhamento Genético (*Seed-and-Extend*):** Durante a evolução biológica ou o sequenciamento em bancada, ocorrem polimorfismos de nucleotídeo único (SNPs), inserções e deleções (*indels*). Se uma leitura (*read*) de tamanho $m = 120$ pb contiver apenas uma base mutada, tanto KMP quanto Rabin-Karp aplicados à leitura inteira retornarão zero casamentos. Contudo, pelo Princípio da

Casa dos Pombos, se uma cadeia de comprimento $m$ possui até $e$ mutações pontuais, ao dividirmos ou extrairmos subcadeias contíguas ($k$-mers) de tamanho $k \leq \lfloor \frac{m-e}{e+1} \rfloor$, pelo menos uma dessas sementes permanecerá 100% livre de mutações. Ferramentas como o BLAST [4] indexam os $k$-mers da consulta usando *rolling hash* (Rabin-Karp na base $d = 4$) para localizar todas as sementes exatas no genoma em $\Theta(n)$ e disparam o alinhamento local de Smith-Waterman restrito às diagonais onde as sementes se acumulam.

# 3 O Paradigma Algorítmico (Engenharia da Solução)

Apresentamos a seguir a especificação formal dos algoritmos utilizando o pacote acadêmico `algorithm2e`, acompanhada dos diagramas estruturais de execução passo a passo.

## 3.1 Pseudocódigos Formais (`algorithm2e`)

**Algoritmo 1:** CASADOR-INGENUO-CADEIAS($T, P$)

**Input:** Texto $T[1..n]$ e padrão $P[1..m]$ sobre o alfabeto $\Sigma$.

**Output:** Deslocamentos válidos $s$ onde $P$ ocorre em $T$.

```
1  n ← T.comprimento
2  m ← P.comprimento
3  para s ← 0 to n − m faça
4  |   j ← 1
5  |   enquanto j ≤ m e T[s + j] == P[j] faça
6  |   |   j ← j + 1
7  |   fim
8  |   se j > m então
9  |   |   imprimir "Padrão ocorre com deslocamento válido " s
10 |   fim
11 fim
```

**Algoritmo 2:** CASADOR-RABIN-KARP$(T, P, d, q)$

**Input:** Texto $T[1..n]$, padrão $P[1..m]$, base $d = |\Sigma|$ e primo $q$.

**Output:** Deslocamentos válidos $s$ onde $P$ ocorre em $T$.

```
1  n ← T.comprimento
2  m ← P.comprimento
3  h ← d^(m−1) mod q
4  p ← 0
5  t_0 ← 0
   // Pré-processamento via Regra de Horner em Θ(m)
6  para i ← 1 to m faça
7  |  p ← (d · p + P[i]) mod q
8  |  t_0 ← (d · t_0 + T[i]) mod q
9  fim
   // Etapa de casamento por janela deslizante (Rolling Hash)
10 para s ← 0 to n − m faça
11 |  se p == t_s então
12 |  |  se P[1..m] == T[s + 1..s + m] então
13 |  |  |  imprimir "Padrão ocorre com deslocamento válido " s
14 |  |  fim
15 |  fim
16 |  se s < n − m então
17 |  |  t_{s+1} ← (d · (t_s − T[s + 1] · h) + T[s + m + 1]) mod q
18 |  |  se t_{s+1} < 0 então
19 |  |  |  t_{s+1} ← t_{s+1} + q
20 |  |  fim
21 |  fim
22 fim
```

**Algoritmo 3:** CALCULAR-FUNCAO-PREFIXO($P$)

**Input:** Padrão $P[1..m]$.

**Output:** Vetor da função de prefixo $\pi[1..m]$.

```
1  m ← P.comprimento
2  Seja π[1..m] um novo vetor
3  π[1] ← 0
4  k ← 0
5  para q ← 2 to m faça
6  |  enquanto k > 0 e P[k + 1] ≠ P[q] faça
7  |  |  k ← π[k]
8  |  fim
9  |  se P[k + 1] == P[q] então
10 |  |  k ← k + 1
11 |  fim
12 |  π[q] ← k
13 fim
14 retorna π
```

**Algoritmo 4:** CASADOR-KMP$(T, P)$

**Input:** Texto $T[1..n]$ e padrão $P[1..m]$.

**Output:** Deslocamentos válidos $s$ onde $P$ ocorre em $T$.

```
1  n ← T.comprimento
2  m ← P.comprimento
3  π ← CALCULAR-FUNCAO-PREFIXO(P)
4  q ← 0                    // Número de caracteres casados no estado atual
5  para i ← 1 to n faça
6      enquanto q > 0 e P[q + 1] ≠ T[i] faça
7          q ← π[q]
8      fim
9      se P[q + 1] == T[i] então
10         q ← q + 1
11     fim
12     se q == m então
13         imprimir "Padrão ocorre com deslocamento " (i − m)
14         q ← π[q]
15     fim
16 fim
```

## 3.2 Diagramas Visuais Passo a Passo

A Figura 1 ilustra o mecanismo algébrico de atualização em $\Theta(1)$ da janela deslizante no algoritmo de Rabin-Karp sobre uma fita de DNA mapeada em base $d = 4$ (A $= 0$, C $= 1$, G $= 2$, T $= 3$).

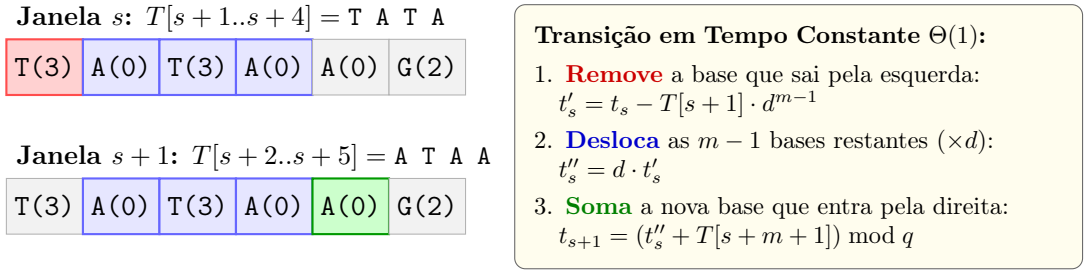



Figura 1: Diagrama de atualização incremental do *Rolling Hash* no algoritmo de Rabin-Karp.

A Figura 2 demonstra o comportamento do KMP e da tabela $\pi$ diante de uma falha após casar $q = 5$ bases do padrão $P =$ `ACAGAC`. Observando o prefixo casado $P_5 =$ `ACAGA`, verifica-se que o maior prefixo próprio de $P$ que também é sufixo de $P_5$ é $P_1 =$ `A` ($\pi[5] = 1$).

Assim, o padrão desliza diretamente $q - \pi[q] = 5 - 1 = 4$ posições sem retroceder o índice de leitura do texto.

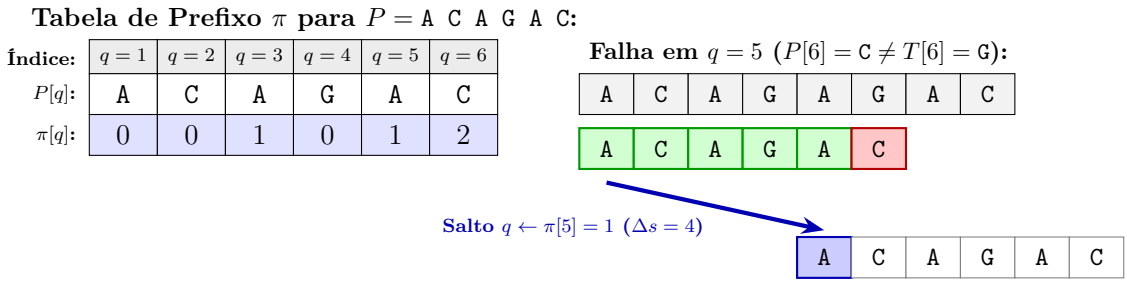



Figura 2: Representação da tabela $\pi$ e do salto determinístico do KMP sem retrocesso no texto $T$.

Por fim, a Figura 3 apresenta o fluxograma da integração dos algoritmos de casamento exato dentro de uma pipeline de alinhamento genético (*Seed-and-Extend*).

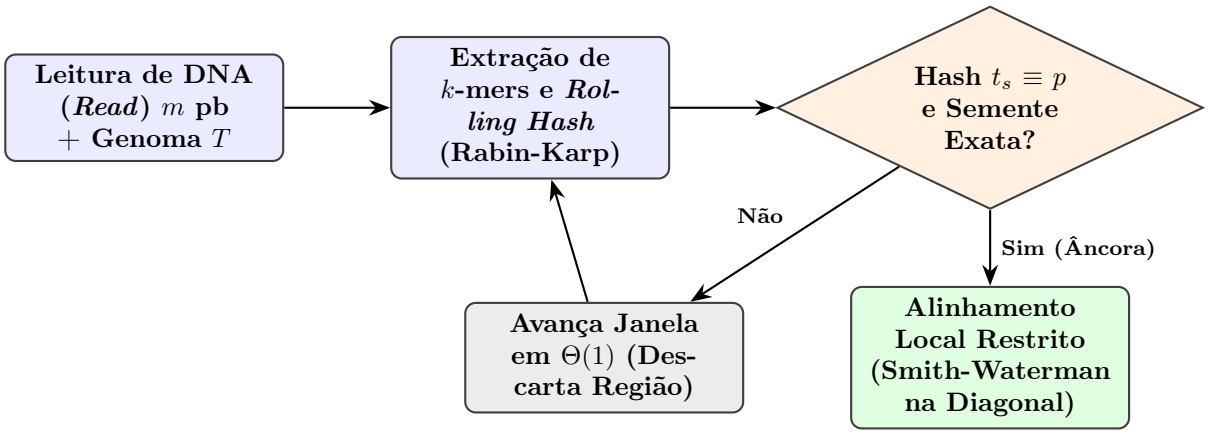



Figura 3: Fluxograma da etapa de *Seeding* via casamento exato acoplada ao alinhamento genético.

# 4 Análise Assintótica Rigorosa (Custos)

Realizamos nesta seção o estudo formal das cotas superiores ($O$), inferiores ($\Omega$) e restritas ($\Theta$) de tempo de processamento e consumo de memória para os três algoritmos.

## 4.1 Complexidade da Busca Ingênua

O Algoritmo 1 não possui etapa de pré-processamento ($T_{\text{pre}} = \Theta(1)$) e utiliza apenas variáveis escalares de controle ($S(n, m) = \Theta(1)$).

- **Melhor Caso – $\Omega(n)$:** Ocorre quando o primeiro caractere do padrão nunca coincide com o texto ($T[s+1] \neq P[1]$ para todo $s$). O laço interno executa exatamente 1

comparação para cada um dos $n-m+1$ deslocamentos, totalizando $n-m+1 = \Omega(n)$ operações.

- **Pior Caso** – $\Theta((n-m+1)m)$**:** Ocorre em cadeias periódicas ou constantes, como $T = \mathtt{A}^n$ e $P = \mathtt{A}^{m-1}\mathtt{C}$. Para cada um dos $n-m+1$ deslocamentos, o laço interno executa exatamente $m$ comparações antes de falhar no último caractere. O número exato de comparações é $(n-m+1)m = \Theta((n-m+1)m)$, que atinge $\Theta(n^2)$ quando $m = \lfloor n/2 \rfloor$.

- **Caso Médio sobre o Alfabeto do DNA** ($|\Sigma| = 4$) – $\Theta(n)$**:** Assumindo bases independentes e uniformemente distribuídas com probabilidade $p_c = 1/|\Sigma| = 1/4$, o número esperado de comparações em um deslocamento arbitrário é limitado pela série geométrica:

$$\mathbb{E}[C_s] = \sum_{j=1}^{m} \left(\frac{1}{|\Sigma|}\right)^{j-1} < \frac{1}{1-1/|\Sigma|} = \frac{4}{3} \approx 1,333 \text{ comparações por posição.} \quad (8)$$

  Pela linearidade da esperança, o total esperado de comparações sobre o texto é $\frac{4}{3}(n-m+1) = \Theta(n)$.

## 4.2 Complexidade do Algoritmo de Rabin-Karp

- **Pré-processamento** – $\Theta(m)$**:** O laço das linhas 6 a 9 do Algoritmo 2 aplica a Regra de Horner exatamente $m$ vezes, executando $\Theta(m)$ operações aritméticas modulares.

- **Melhor Caso de Busca** – $\Omega(n-m+1)$**:** Se o padrão não ocorre no texto e não há colisões espúrias, cada janela realiza apenas a comparação inteira $p == t_s$ e a atualização da Equação (4) em $\Theta(1)$, resultando em $\Theta(n-m+1)$.

- **Pior Caso de Busca** – $\Theta((n-m+1)m)$**:** Se todos os deslocamentos tiverem $t_s \equiv p \pmod{q}$ (por exemplo, $T = \mathtt{A}^n$ e $P = \mathtt{A}^m$, ou escolha patológica de $q$), a verificação caractere a caractere de custo $\Theta(m)$ é acionada em todas as $n-m+1$ janelas.

- **Caso Médio de Busca** – $O(n+m)$**:** Supondo que o módulo $q$ atue como uma função de dispersão uniforme sobre $\mathbb{Z}_q$, a probabilidade de dois hashes distintos colidirem modulo $q$ é $1/q$. Com $\nu$ ocorrências válidas no texto, o número esperado de colisões espúrias é $O((n-m+1)/q)$. Cada verificação custa $O(m)$, fornecendo o tempo esperado total:

$$\mathbb{E}[T_{\text{busca}}] = O(n) + O\left(m\left(\nu + \frac{n}{q}\right)\right) \quad (9)$$

Escolhendo um primo $q \geq m$ (na prática $q = 10^9 + 7 \gg n$), o termo $mn/q$ torna-se inferior a $O(n)$, garantindo tempo esperado $\Theta(n + m)$ quando $\nu = O(1)$.

- **Complexidade de Espaço** – $\Theta(1)$**:** O algoritmo armazena apenas os inteiros escalares $p, t_s, h, d, q$, consumindo memória auxiliar constante.

## 4.3 Complexidade Amortizada Rigorosa do Algoritmo KMP

A grande virtude teórica do KMP reside no fato de que seu melhor caso, caso médio e pior caso coincidem assintoticamente em $\Theta(n + m)$.

**Teorema 4.1** (Custo Linear $\Theta(m)$ de CALCULAR-FUNCAO-PREFIXO)**.** *O Algoritmo 3 calcula o vetor $\pi[1..m]$ em tempo estritamente $\Theta(m)$ no pior caso.*

*Demonstração.* Utilizamos o **método da função potencial** (ou método agregado) sobre a variável inteira $k$:

1. Inicialmente, na linha 4, temos $k = 0$, e o invariante $0 \leq k < q$ mantém-se em todas as iterações, garantindo que $k \geq 0$ nunca assume valores negativos.

2. O único ponto do algoritmo onde $k$ aumenta é na linha 10 ($k \leftarrow k+1$), que é executada no máximo uma vez por iteração do laço **para** externo ($q = 2, \ldots, m$). Logo, ao longo de toda a execução do procedimento, $k$ sofre no máximo $m-1$ incrementos unitários.

3. No interior do laço **enquanto** (linha 7), a atribuição $k \leftarrow \pi[k]$ substitui $k$ por um valor estritamente menor, pois pela definição da função de prefixo temos $\pi[k] < k$, implicando que cada iteração do laço **enquanto** reduz $k$ em pelo menos 1 unidade ($\Delta k \leq -1$).

4. Como $k$ parte de 0, nunca fica negativo e sobe no máximo $m-1$ unidades no total, o laço **enquanto** não pode executar mais do que $m-1$ vezes somando todas as $m-1$ iterações do laço **para**.

Consequentemente, o custo total de todas as comparações e atribuições é limitado superiormente por $2(m-1)$ operações e inferiormente pelas $m-1$ iterações do laço externo, provando que $T_{\text{pre}}(m) = \Theta(m)$. □

**Teorema 4.2** (Custo Linear $\Theta(n)$ de CASADOR-KMP)**.** *A etapa de busca do Algoritmo 4 executa em tempo estritamente $\Theta(n)$ no pior caso, realizando no máximo $2n$ comparações de caracteres.*

*Demonstração.* Aplicamos o mesmo argumento potencial sobre o estado $q$ (número de caracteres casados):

1. O estado $q$ inicia em 0 e satisfaz $0 \leq q \leq m$ durante todo o laço principal ($i = 1, \ldots, n$).

2. Em cada uma das $n$ iterações do laço **para**, a condicional da linha 9 incrementa $q$ em no máximo 1 unidade ($q \leftarrow q + 1$). Portanto, a soma total de aumentos de $q$ em toda a busca é no máximo $n$.

3. Cada iteração do laço **enquanto** (linha 7, $q \leftarrow \pi[q]$) e cada redefinição após casamento completo (linha 13) reduz $q$ em pelo menos 1 unidade, visto que $\pi[q] \leq q - 1$.

4. Como $q \geq 0$ sempre se verifica, o número total acumulado de execuções do corpo do laço **enquanto** ao longo de todas as $n$ posições do texto é no máximo $n$.

Somando no máximo $n$ comparações bem-sucedidas (ou falhas em $q = 0$) na linha 8 com no máximo $n$ comparações que falham no laço **enquanto**, o algoritmo realiza no máximo $2n$ comparações de caracteres e no mínimo $n$ comparações, resultando em $T_{\text{busca}}(n) = \Theta(n)$ e custo global $\Theta(n + m)$. O consumo de memória auxiliar é $\Theta(m)$ devido à alocação do vetor $\pi[1..m]$. □

# 5 Estudo de Caso e Implementação Prática

## 5.1 Arquitetura do Repositório e Ambiente de Testes

Todo o código-fonte em C++17, os conjuntos de dados genômicos em formato FASTA e os scripts de automação de gráficos encontram-se disponíveis publicamente no repositório oficial da equipe no GitHub:

`https://github.com/ImArthz/Aeds2_casamento_de_cadeias.git`

A solução foi estruturada nos módulos `include/casamento_cadeias.hpp` e `src/casamento_cadeias.cpp` (contendo as implementações instrumentadas de busca ingênua, Rabin-Karp, KMP e o motor de *seeding* de $k$-mers), `src/fasta_io.cpp` (leitor de arquivos FASTA e gerador de sequências sintéticas de DNA) e `src/main.cpp`. O código foi compilado com otimização máxima (`-std=c++17 -O3`) para avaliação fiel ao nível de máquina.

Para validar o comportamento em dados biológicos reais, coletamos do banco de dados **NCBI RefSeq** duas sequências de referência:

1. **Genoma completo do SARS-CoV-2 (`NC_045512.2`):** Isolado Wuhan-Hu-1, possuindo $n = 29.903$ pares de base (pb), encerrado por uma cauda poli-A real de 33 adeninas consecutivas na extremidade $3'$.

2. **Transcrito humano *TP53* (`NM_000546.6`):** RNA mensageiro da proteína supressora tumoral p53 humana, com $n = 2.512$ pb.

## 5.2 Experimento 1: Busca de Padrões Biológicos no Genoma do SARS-CoV-2

Executamos os algoritmos sobre os 29.903 pb do SARS-CoV-2 buscando quatro alvos biológicos distintos: (i) o motivo promotor **TATA-box** (`TATAAA`, $m = 6$, consenso JASPAR `MA0108.2`); (ii) o iniciador diagnóstico de RT-PCR da OMS **RdRp_SARSr-F2** (`GTGAAATGGTCATGTGTGGCGG`, $m = 22$); (iii) o **sítio de clivagem da furina** na glicoproteína Spike (`CCTCGGCGGGCACGTAGT`, $m = 18$); e (iv) um segmento de **cauda poli-A** ($\texttt{A}^{25}$, $m = 25$). Os resultados medidos encontram-se na Tabela 1 e na Figura 4.

| **Padrão Biológico** | **Algoritmo** | **Hits** | **Comparações** | **Colisões** | **Tempo Total (ms)** |
|---|---|---|---|---|---|
| Motif TATA-box ($m = 6$) | Ingênuo | 20 | 42.688 | 0 | 0,1137 |
| | Rabin-Karp ($q = 10^9 + 7$) | 20 | 120 | 0 | 0,1226 |
| | Rabin-Karp ($q = 101$) | 20 | 412 | 197 | 0,1254 |
| | KMP | 20 | 42.503 | 0 | 0,1336 |
| Primer OMS RdRp-F2 ($m = 22$) | Ingênuo | 1 | 38.495 | 0 | 0,0818 |
| | Rabin-Karp ($q = 10^9 + 7$) | 1 | 22 | 0 | 0,1212 |
| | Rabin-Karp ($q = 101$) | 1 | 386 | 288 | 0,1247 |
| | KMP | 1 | 38.403 | 0 | 0,1084 |
| Sítio Furina Spike ($m = 18$) | Ingênuo | 1 | 36.685 | 0 | 0,0693 |
| | Rabin-Karp ($q = 10^9 + 7$) | 1 | 18 | 0 | 0,1245 |
| | Rabin-Karp ($q = 101$) | 1 | 366 | 285 | 0,1244 |
| | KMP | 1 | 36.371 | 0 | 0,0923 |
| Cauda Poli-A ($m = 25$) | Ingênuo | 9 | 43.065 | 0 | 0,0906 |
| | Rabin-Karp ($q = 10^9 + 7$) | 9 | 225 | 0 | 0,1217 |
| | Rabin-Karp ($q = 101$) | 9 | 660 | 304 | 0,1255 |
| | KMP | 9 | 41.751 | 0 | 0,1562 |

Tabela 1: Métricas empíricas de busca exata no genoma real do SARS-CoV-2 (`NC_045512.2`, 29.903 pb).

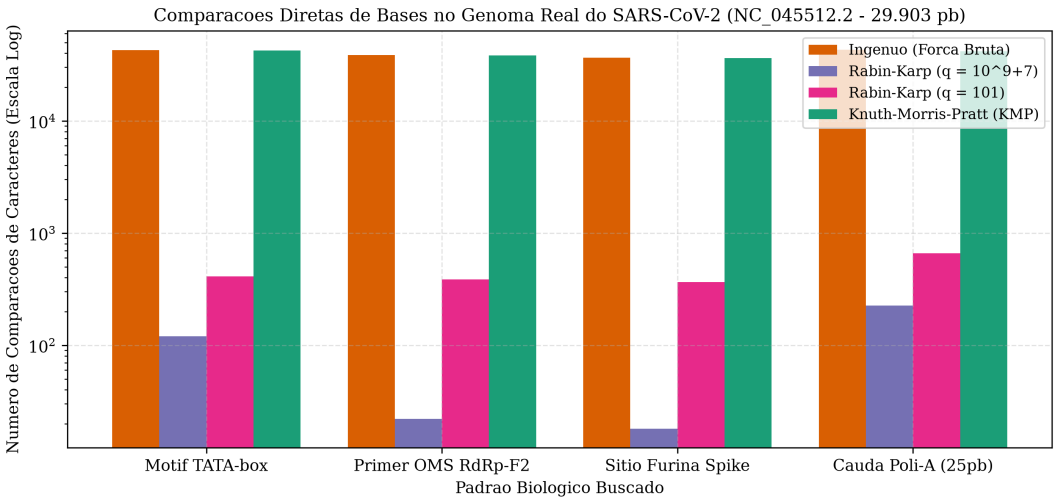



Figura 4: Comparações diretas de caracteres realizadas por cada algoritmo sobre o genoma do SARS-CoV-2.

Adicionalmente, testamos na função `seeding_rabin_karp` a recuperação de uma leitura simulada de 120 pb extraída da posição 21.562 (início do gene *Spike*) na qual introduzimos artificialmente duas mutações pontuais (SNPs) nos índices 35 e 88. Ao buscar a leitura inteira de 120 pb com o KMP, o resultado foi de 0 ocorrências devido às duas bases divergentes. Em contrapartida, a etapa de *seeding* via *rolling hash* de Rabin-Karp ($k = 15$) localizou instantaneamente 76 âncoras exatas preservadas nos segmentos intactos da leitura, apontando exatamente para o deslocamento original $s = 21.562$ do genoma viral.

## 5.3 Experimento 2: Escalabilidade Linear com Entradas Progressivas ($N$)

Na Figura 5, avaliamos o comportamento com entradas progressivas de DNA pseudo-aleatório variando de $N = 10.000$ pb até $N = 5.000.000$ pb (5 Mpb), fixando $M = 64$ pb.

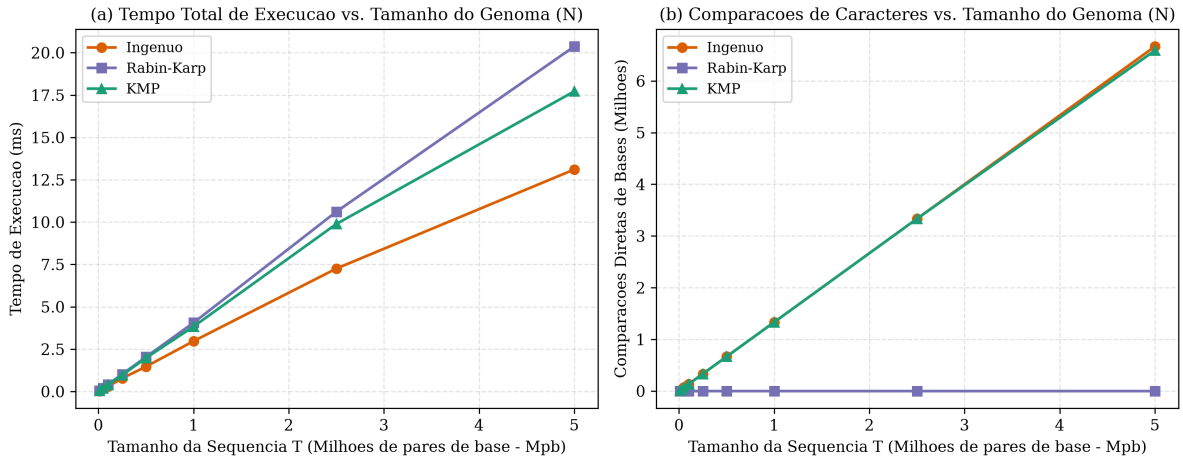



Figura 5: Evolução do tempo de execução (a) e do número de comparações (b) com entradas progressivas até 5 Mpb.

Os dados confirmam visual e numericamente a linearidade $\Theta(n)$ dos três métodos no caso médio uniforme. Para $N = 5.000.000$ pb, tanto a busca ingênua quanto o KMP efetuam cerca de $6,66 \times 10^6$ comparações de caracteres ($\approx 1,33 \cdot N$, cravando a previsão teórica de $\frac{4}{3}N$ deduzida na Seção 4.1).

## 5.4 Experimento 3: Pior Caso Patológico em DNA Repetitivo e Impacto do Primo $q$

Para expor a falha catastrófica do algoritmo ingênuo em regiões homopoliméricas de DNA, submetemos os algoritmos a uma sequência repetitiva poli-A de comprimento $N = 100.000$ pb com padrões de tamanho $M \in \{50, 100, 250, 500, 1000, 2000\}$ da forma $\texttt{A}^{M-1}\texttt{C}$.

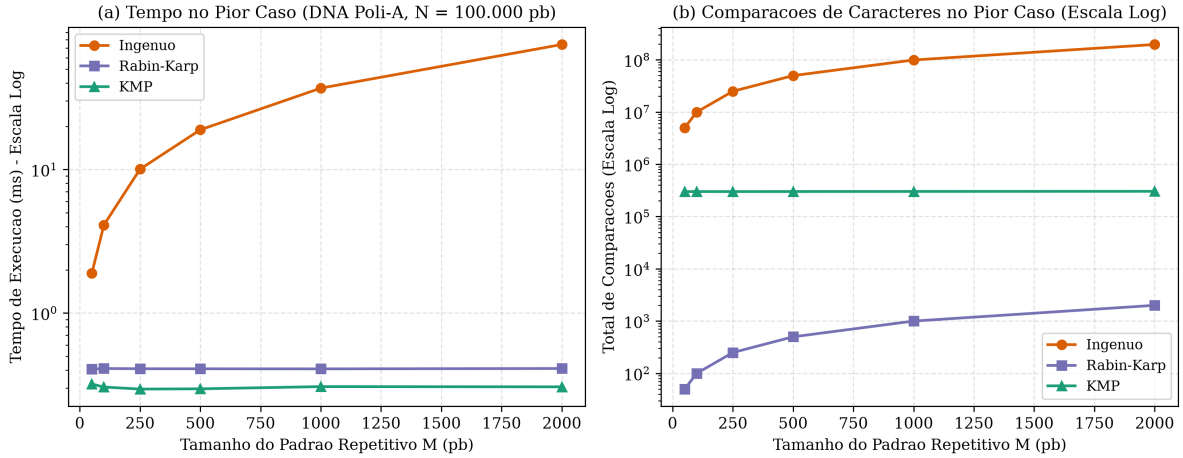



Figura 6: Degradação quadrática $\Theta(nm)$ da busca ingênua versus estabilidade linear $\Theta(n+m)$ do KMP e Rabin-Karp em DNA repetitivo.

Os resultados na Figura 6 são contundentes: quando $M = 2.000$, a busca ingênua realiza exatas 196.002.000 comparações e consome $74,17$ ms. O algoritmo KMP, consultando o vetor $\pi$, realiza apenas 303.993 comparações e finaliza a busca em $0,306$ ms (um ganho de velocidade de **242 vezes**), mantendo o tempo de busca inalterado ($\approx 0,30$ ms) independentemente do aumento de $M$.

Na Figura 7, investigamos o impacto da escolha do módulo primo $q$ sobre as colisões espúrias no Rabin-Karp ($N = 500.000$ pb, $M = 32$ pb). Quando utilizamos um primo pequeno ($q = 13$), ocorrem 38.899 falsos positivos de hash ($\approx N/13$), forçando 51.802 comparações extras de caracteres e elevando o tempo de busca para $2,53$ ms. Ao escalar o primo para $q = 10^9 + 7$, o número de colisões espúrias cai para zero absoluto, restringindo as comparações de caracteres às 32 bases da única ocorrência real.

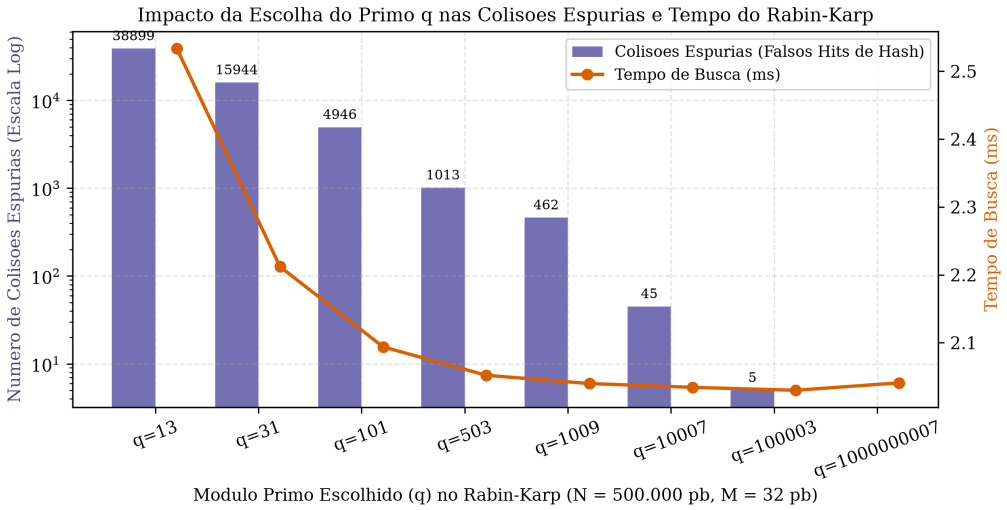



Figura 7: Decaimento logarítmico das colisões espúrias e redução do tempo de busca no Rabin-Karp em função do primo $q$.

## 5.5 Discussão Crítica: Gargalos de Hardware e de Linguagem em C++17

A análise conjunta dos resultados práticos revela nuances de arquitetura de computadores que a notação assintótica pura omite:

1. **Latência da Instrução de Divisão/Módulo na ALU (`idiv`):** Na Figura 5(a), observa-se que, em DNA uniforme aleatório ($N = 5$ Mpb), o Rabin-Karp levou $20,36$ ms frente a $17,71$ ms do KMP e $13,10$ ms do algoritmo ingênuo, mesmo tendo efetuado apenas 64 comparações diretas de caracteres! Isso ocorre porque, para cada um dos 5 milhões de deslocamentos, o Rabin-Karp executa a operação `% q` sobre inteiros de 64 bits. Em arquiteturas x86_64, o operador `%` com divisor variável

compila para a instrução `idiv`, cuja latência na unidade lógico-aritmética (ALU) varia de 20 a 40 ciclos de clock, enquanto a instrução `cmp` de 1 byte na memória cache L1 executada pela busca ingênua consome apenas 1 ciclo de clock.

2. **Semântica do Operador % para Dividendos Negativos em C/C++:** Diferentemente da definição matemática de mod$q$ adotada no Cormen (que sempre retorna valores em $[0, q-1]$), o padrão ISO C++ define `a % b` com o mesmo sinal do dividendo `a`. Na subtração do dígito mais significativo `t - char_sai * h`, o resultado frequentemente se torna negativo, o que corromperia todos os hashes subsequentes caso não incluíssemos a correção explícita:

```
int64_t char_sai = static_cast<unsigned char>(T[s]);
int64_t char_entra = static_cast<unsigned char>(T[s + m]);
t = (d * (t - char_sai * h) + char_entra) % q;
if (t < 0) {
    t += q; // Ajusta o resto truncado do C++ para o representante positivo em Z_q
}
```

Listing 1: Tratamento de overflow e resto negativo no Rolling Hash em C++17

3. **Prevenção de *Integer Overflow* e Conversão de Índices Base-0:** Caso utilizássemos o tipo padrão `int` (32 bits, limite $2,14 \times 10^9$) com $q = 10^9 + 7$ e $d = 256$, o produto intermediário $d \cdot (t - \text{char_sai} \cdot h)$ alcançaria a ordem de $2,56 \times 10^{11}$, provocando *signed integer overflow* (comportamento indefinido em C++). O uso estrito de `int64_t` (limite $9,22 \times 10^{18}$) elimina esse gargalo. No KMP, a adaptação da notação base-1 do Cormen (`k = pi[k]`) para vetores base-0 em C++ exigiu mapear o recuo de estado para `k = pi[k - 1]`, mantendo o vetor $\pi$ contíguo na cache L1 (256 bytes para $M = 64$).

# 6 Conclusões

O estudo teórico e experimental desenvolvido neste trabalho permitiu confrontar a elegância matemática dos algoritmos de casamento de cadeias com a realidade física de sua execução sobre dados genômicos. Ficou demonstrado que cada abordagem possui um nicho de superioridade bem delimitado:

- **Onde o KMP é imbatível:** Na busca determinística de um único padrão sobre alfabetos pequenos ($|\Sigma| = 4$ no DNA) ou textos sujeitos a repetições periódicas (caudas poli-A, repetições em tandem). O KMP oferece garantia estrita de tempo

$\Theta(n+m)$ no pior caso, sem depender de escolhas de números primos e realizando apenas comparações leves na cache L1.

- **Onde o Rabin-Karp se destaca:** Na busca simultânea de **múltiplos padrões** de comprimento fixo (como na indexação de dezenas de milhares de $k$-mers em uma tabela hash durante a etapa de *seeding* de alinhadores genéticos) e no casamento de padrões bidimensionais. Enquanto o KMP exigiria construir um autômato de Aho-Corasick com alto consumo de memória para múltiplos padrões, o Rabin-Karp calcula o hash da janela do texto uma única vez em $\Theta(1)$ e consulta o dicionário de sementes instantaneamente.

- **Onde o paradigma falha:** Tanto o KMP quanto o Rabin-Karp puro falham quando o problema exige tolerância direta a mutações pontuais (SNPs) ou inserções/deleções (*indels*) ao longo de toda a cadeia sem subdivisão em sementes. Nesses cenários aproximados, ou quando o mesmo genoma estático de referência será consultado milhões de vezes, o paradigma de varredura on-line $\Theta(n)$ cede espaço para estruturas de indexação prévia do texto (como Suffix Arrays e a Transformada de Burrows-Wheeler com FM-Index [5], que reduzem a busca para $O(m)$) combinadas com programação dinâmica local.

# Referências


[1] CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. **Algoritmos: Teoria e Prática**. 3. ed. Rio de Janeiro: Elsevier / GEN LTC, 2012. Cap. 32 (Casamento de Cadeias), p. 719–745.

[2] KNUTH, D. E.; MORRIS, J. H.; PRATT, V. R. Fast pattern matching in strings. **SIAM Journal on Computing**, v. 6, n. 2, p. 323–350, 1977. DOI: `https://doi.org/10.1137/0206024`.

[3] KARP, R. M.; RABIN, M. O. Efficient randomized pattern-matching algorithms. **IBM Journal of Research and Development**, v. 31, n. 2, p. 249–260, 1987. DOI: `https://doi.org/10.1147/rd.312.0249`.

[4] ALTSCHUL, S. F.; GISH, W.; MILLER, W.; MYERS, E. W.; LIPMAN, D. J. Basic local alignment search tool. **Journal of Molecular Biology**, v. 215, n. 3, p. 403–410, 1990. DOI: `https://doi.org/10.1006/jmbi.1990.9999`.

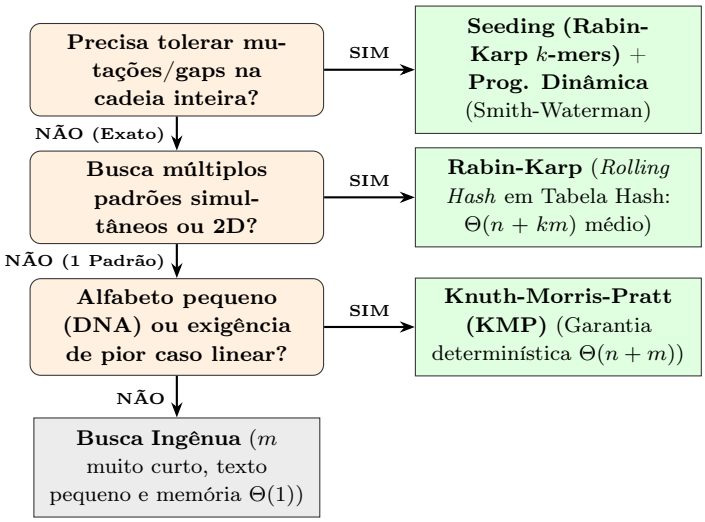
[5] LI, H.; DURBIN, R. Fast and accurate short read alignment with Burrows-Wheeler transform. **Bioinformatics**, v. 25, n. 14, p. 1754–1760, 2009. DOI: https://doi.org/10.1093/bioinformatics/btp324.

[6] MENDONÇA, A. O.; FONSECA, I. X.; MARTINS, T. A. S.; GUERREIRO, T.; TAVARES, M. **Implementação e Análise Experimental de Algoritmos de Casamento de Cadeias (Rabin-Karp e KMP) em Sequências Genômicas**. Repositório público de código-fonte e experimentos no GitHub, 2026. Disponível em: https://github.com/ImArthz/Aeds2_casamento_de_cadeias.git. Acesso em: 02 nov. 2026.

# Apêndice: Cheat Sheet – Casamento de Cadeias (Cap. 32)

**a) Definição do Paradigma em Uma Frase:**
Localizar todas as posições de ocorrência de um padrão $P[1..m]$ dentro de um texto $T[1..n]$ em tempo linear $\Theta(n+m)$, substituindo re-comparações redundantes por impressões digitais aritméticas (*Rolling Hash* no Rabin-Karp) ou transições de estado baseadas em auto-sobreposição de prefixos (Tabela $\pi$ no KMP).

## b) Diagrama de Decisão: "Quando Utilizar Esta Abordagem?"




## c) Tabela Resumo das Complexidades de Tempo e Espaço

| Algoritmo | Pré-processamento | Melhor Caso ($\Omega$) | Caso Médio ($\Theta$) | Pior Caso ($O/\Theta$) | Espaço Auxiliar |
|---|---|---|---|---|---|
| **Ingênuo (Força Bruta)** | 0 (Nenhum) | $\Omega(n)$ | $\Theta(n)$ | $\Theta((n-m+1)m)$ | $\Theta(1)$ |
| **Rabin-Karp** | $\Theta(m)$ | $\Omega(n-m+1)$ | $\Theta(n+m)$ | $\Theta((n-m+1)m)$ | $\Theta(1)$ |
| **Knuth-Morris-Pratt (KMP)** | $\Theta(m)$ | $\Omega(n+m)$ | $\Theta(n+m)$ | $\Theta(n+m)$ | $\Theta(m)$ |

## d) Esqueleto Base da Técnica (Rabin-Karp & KMP em Base-0)

```
// 1. ESQUELETO KMP: Construcao de pi em Theta(m) + Busca sem retrocesso em Theta(n)
vector<int> pi(m, 0);
for (int q = 1, k = 0; q < m; ++q) {
    while (k > 0 && P[k] != P[q]) k = pi[k - 1]; // Recua pelo maior sufixo-prefixo
    if (P[k] == P[q]) k++;
    pi[q] = k;
}
for (int i = 0, q = 0; i < n; ++i) {
    while (q > 0 && P[q] != T[i]) q = pi[q - 1]; // i nunca retrocede!
    if (P[q] == T[i]) q++;
    if (q == m) { reportar_match(i + 1 - m); q = pi[q - 1]; }
}

// 2. ESQUELETO RABIN-KARP: Atualizacao O(1) do Rolling Hash via Aritmetica Modular
// Pre-calculo: h = d^(m-1) mod q; p e t calculados via Regra de Horner nos primeiros m
    chars
for (int s = 0; s <= n - m; ++s) {
    if (p == t && T.compare(s, m, P) == 0) reportar_match(s); // Filtra colisao espuria
    if (s < n - m) {
        t = (d * (t - T[s] * h) + T[s + m]) % q;
        if (t < 0) t += q; // Garante residuo positivo em Z_q
    }
}
```